2. Source Coding

2.1. The Basics

The design of encoders and decoders – such as in Figure 1.1 – is performed in two steps (both in theory and in practice) in order to accomplish the two goals mentioned before. The purpose of **source coding** is to improve the efficiency of the channel usage, to **compress** the content of the source without any loss of information. Transmitting information through noisy channels (with as less distortion as possible) is the field of **channel coding**. As a result, when source coding algorithms are mentioned, it is presumed that the channels have **no noise**. Thus Figure 1.1 is modified accordingly and becomes Figure 2.1.

In most practical situations **binary** channels are used – which means only "0" and "1" symbols are available for storing and transmitting – so the following part discusses coding methods for such binary channels. It is also presumed that only one binary symbol can be transmitted at a time, and that the **cost** of transmission is proportional to the amount of symbols sent (i.e. to the number of "channel uses"). In the process of encoding a **code word** (a string of "0" and "1" symbols) is assigned to every source symbol. This assignment (or mapping) is referred to as **code**, for short.

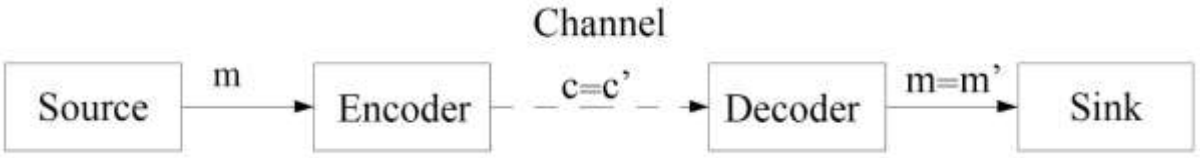


Figure 2.1: *Basic model of source coding*

The **main idea** behind source coding is that shorter code words are used for frequent source symbols and the longer ones are assigned to less frequent symbols. With this we can decrease the cost paid for the channel usage (for transmission sessions "long enough"). A code that uses code words of different lengths is called a **variable-length code**. For such codes we define L(A), **the average code word length** as

$$L(A) = \sum_i p_i l_i$$

where l_i is the length of the code word for the i-th symbol, and p_i is its probability. Sending one code word through the channel has an average cost of L(A), so our task will be to minimize this L(A).

A code is said to be **decodable** if a unique source symbol sequence can be found for every code word sequence generated. This means we cannot have any doubts when decoding. A sufficient[7] condition for a decodable code is that it is a **prefix** code, i.e.

- for every source symbol there is a unique code word, and
- no code word is a prefix of any other code word.

Example 2.1
Coding a source of 3 symbols

Symbol	*Prefix code words*	*Non-prefix code words*
"a"	"0"	"1"
"b"	"10"	"10"
"c"	"11"	"11"

[7] It is not a necessary condition, e.g. *postfix* codes are also decodable.

When using a non-prefix code, the decoder at the recipient's side is in trouble if it receives a symbol sequence such as "111111". It is caused by the code not being decodable.

It is sufficient because the boundaries of the code words can be specified clearly by knowing every valid code word at the recipient's side, and the original source symbol sequence can be decoded as they are all distinct.

A code is **undecodable** if decoding a message is ambiguous, i.e. it is not possible to clearly separate the code words of the message. In some cases, however, this property can be found to be useful: such an example can be found in the Hungarian history, in the plot against queen Gertrudis during the 13th century (a.k.a. the story of Bánk bán). When the plotting nobles have asked János Merániai, the archbishop of Esztergom, for his opinion and approval for their plans, they received a response message containing no punctuation:

"REGINAM OCCIDERE NOLITE TIMERE BONUM EST SI OMNES CONSENTIUNT EGO NON CONTRADICO"

This can be (roughly) translated as:

FEAR NOT TO KILL THE QUEEN IF OTHERS CONSENT I DO NOT OPPOSE

As a result, this message can be read in two ways, having two totally different and contradicting meanings. If we separate it into 3 "code words" we get a consent:

Fear not to kill the Queen. | *If others consent,* | *I do not oppose.*

But if we separate it into 5 parts, then it reads:

(have) *Fear.* | *Not to kill the Queen.* | *If others consent,* | *I do not.* | (I) *oppose.*

This way, if the plot had been successful, he could have appeared to be on their side, but as the plot has failed (the queen was killed, but the king returning to the country executed most of the rebels), he was able to state that he had no intent of helping them, quite the contrary.

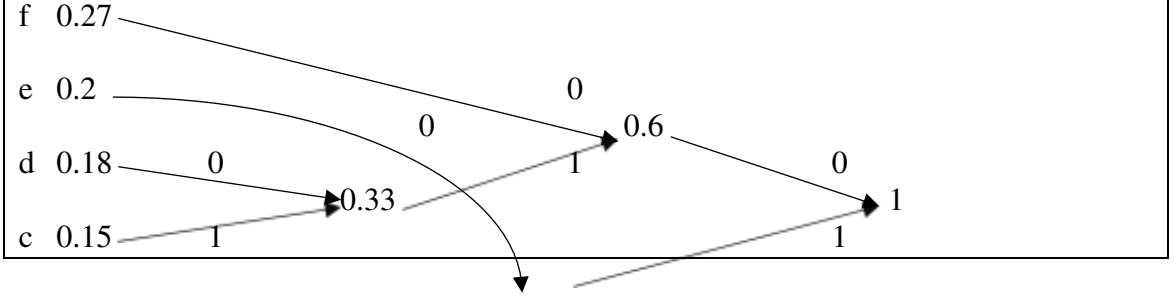
2.2. Huffman Coding

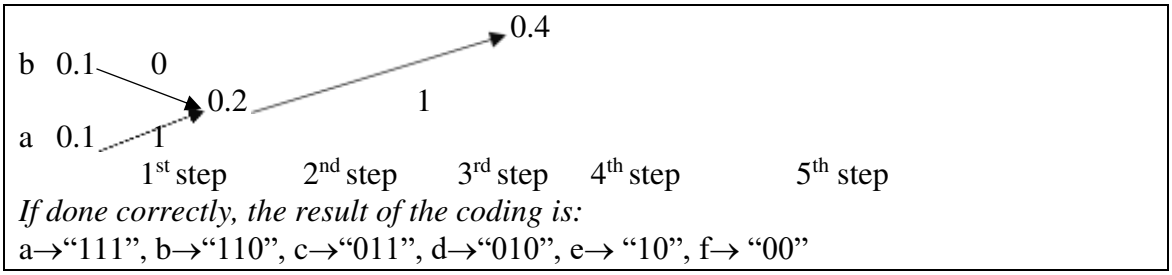
The Huffman code is a prefix, variable-length code originating from 1952. To build such a code we need to know (e.g. from statistical measurements) the probabilities of the source symbols. The code words are constructed in the following procedure:

- Sort the symbols based on their probabilities into decreasing order.
- Combine the two lowest probability (p_1, p_2) symbols into a new "compound" symbol, which has the probability $p_3 = p_1 + p_2$.

We continue this until only one symbol is left with a probability of 1. Then we have a binary tree for which each branching is where we combined two symbols, and its leaves are the original symbols. Code words are then assigned by reading the tree from right to left, adding a "1" to the end of the code word if we go "down", a "0" for going "up".

Example 2.2
Source of five symbols A = {a, b, c, d, e, f} = {0.1, 0.1, 0.15, 0.18, 0.2, 0.27}





If done correctly, the result of the coding is:

a→"111", b→"110", c→"011", d→"010", e→ "10", f→ "00"

It is easy to see that – because of the "binary tree" building rule – the resulting code is prefix, so it is also decodable.

In **hardware** implementations of Huffman coders (e.g. telefax), the encoder stores both the code words and the binary numbers representing their lengths in a ROM (e.g. on the first 10 bits we store the length, and on the next 22 bits the code word itself). The code words are sent to the channel by a shift register managed by a controller, based on the length of the current word. In Figure 2.2 an outline of one possible implementation is shown.

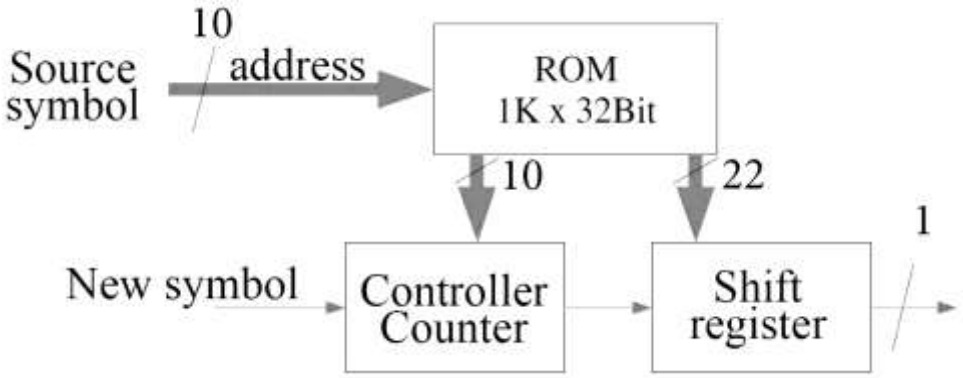


Figure 2.2: *Hardware outline of a Huffman coder*

The decoder uses the prefix property: if the received segment fits only one valid code word, then we have reached the end of the word. This can be achieved using an associative memory. One specific practical application of Huffman coding is the "traditional" (Group 3) black-and-white telefax standard.

2.3. Source Coding Theorems

Shannon's first theorem, also known as the **source coding theorem** (or noiseless-coding theorem) states the theoretical possibilities and limits of source coding.

Let A be a stationary discrete memoryless source, and to every a_i symbol assign a binary code word of length l_i. Then there *exists* a decodable source code such that

$$L(A) < H(A) + 1$$

but there *exists none* that satisfies

$$L(A) < H(A)$$

Hence,

$$H(A) \leq L(A) < H(A) + 1$$

can be achieved, where the left hand side is the "negative", the right hand side is the "positive" statement of the theorem. We can notice that source entropy serves as a basic lower bound on source compression, but it can be approached within 1-bit distance with proper coding. (The previously mentioned Huffman coding has this property.) The proof of this statement and of the theorem can be found in corresponding literature.

From Shannon's theorem we can define the **efficiency** h of a code as

$$h = H(A)/L(A)\ [\%]$$

Efficiency varies between 0 and 100%, in ideal cases it can reach 100%. It can be noticed that for Huffman coding such a case is when all symbol probabilities are a negative power of 2. A code of 100% efficiency is called an **optimal code**.

For sources that contain only a few symbols or that have an uneven source distribution, even the average code word length $H(A) + 1$ might prove to be highly inefficient, e.g. for A = {0.001, 0.999}, $h \approx 0.011 / 1 = 1.1\%$. In such situations we can take N consecutive symbols and unite them into a single symbol, extending the source N times. The number of symbols in this **extended source** will be $|A_{ext}| = |A|^N$ and the probabilities for these symbols will be the products of the original symbol probabilities (e.g. $P(a_1, a_3, a_1) = p_1^2 \cdot p_3$ etc.) for memoryless sources. It can be shown that the entropy of an N-fold extended memoryless source is N times the source entropy[8]. If we apply the source coding theorem on the extended source and then divide by block size N, then it can be noticed that the theoretical limit can be approached to any arbitrary degree of closeness by increasing the value of N due to source extension:

$$H(A_{ext}) \leq L(A_{ext}) < H(A_{ext}) + 1$$

$$N \cdot H(A_{orig}) \leq L(A_{ext}) < N \cdot H(A_{orig}) + 1$$

$$H(A_{orig}) \leq L(A_{orig}) < H(A_{orig}) + \frac{1}{N}$$

where $H(A_{orig})$ is the entropy and $L(A_{orig})$ is the average code word length for the original source. This result is the **source extension theorem**.

Example 2.3
Given the $A = \{a_0, a_1\} = \{0.2, 0.8\}$ *source, H(A) = 0.721 bits.*
Using two code words of length 1 i.e. "0" and "1":
$L(A) = 1$, **h = 72.1**%.
Using a Huffman-code on the twofold extended source:
e.g. $P(a_0\, a_0) = 0.04$, $P(a_0\, a_1) = 0.16$ *etc.*
$L(A \times A) = 1.56$, $H(A \times A) = 1.44$ *bits, so* **h = 92.4**%

A drawback of the source extension method is that the cardinality of the source grows exponentially (which results in an expensive and slow hardware), moreover, in cases when the sources are not memoryless[9], it is troublesome to create the source distribution statistics. So when we extend a source or if we do not have full knowledge of the source distribution, it might be necessary to observe the source during runtime (when encoding) and modify the basic source probabilities used by the Huffman code accordingly.

It can be shown that Huffman coding is not sensitive to minor errors in source probabilities, i.e. using estimated values will not result in a considerable loss of code efficiency, compared to using the accurate probabilities.

2.4. Lempel-Ziv Coding

Lempel-Ziv coding (a.k.a. LZ code or dictionary code) uses a completely different approach from Huffman coding. It serves as a basis for numerous **compression** software (e.g. UNIX compress). Its greatest advantage is that it requires no preliminary knowledge of the source

[8] Just like in the case of H(A, B) = H(A) + H(B) for independent sources A, B. If A is memoryless, then it means that two consecutive symbols (that form a composite symbol) are independent: $H(A \times A) = H(A) + H(A) = 2H(A)$.
[9] e.g. natural language texts

distribution. It only needs to know the number of source symbols used. It can be shown that for *long enough messages* the average code word length approaches the source entropy, i.e. the code is optimal.

When encoding, LZ coding constructs a linked list structure that contains trees rooted in the initial symbols (forming a "forest"). This forest is the dictionary referred to by the code words. The main steps of the algorithm:

1. Initialize the dictionary, pointer *n* and *m* address registers
   - The columns of the dictionary are the address, the pointer and the symbol. At start it has $|A| + 1$ rows, the content of the i-th row is $(i, 0, a_{i-1})$. The 0th row is (0, 0, nil).
   - At start n = 0, m = $|A| + 1$, i.e. the address register points to the dictionary's end.
2. Read the next symbol *x*, exit if no such symbol exists
3. *If* (n, x) is already an element of the dictionary at any address,
   then n = address(n, x)
   else SEND(n)
   store (n, x) at address m
   m = m+1
   n = address(0, x)
4. Return to step 2.

Of course, instead of SEND, the command STORE can also be used, if that is our purpose. When the end of the symbol sequence is reached, the content of register *n* has to be sent too.

It can be seen that the algorithm tries to find a subsequence in the incoming symbol sequence that is already stored in the dictionary. If an incoming symbol does not match the end of the previously found substring or the string ends, only then the address of the end of the substring is sent (which, having the dictionary, will identify the string). Then the pointer is set to the root: n = address(0, x). So next time, when the same symbols sequence appears (e.g. it is a frequent expression in a text) the algorithm will be able to match it for *one more symbol longer* without extending the dictionary or sending additional information. Given an encoding process long enough, the whole content of The Tragedy of Man[10] could be stored with a single *n*.

How can we describe this algorithm using the notions of Section 2.1? As it generates only one code word from an arbitrary long input symbol sequence, we can say that it extends the source to an arbitrary level and the missing source distribution is resolved by the adaptive modification applied when encoding.

Decoding

From the received code word sequence (address/pointer sequence) and the known source alphabet it is possible to reconstruct both the dictionary and the encoded symbol sequence.

1. Initialize the dictionary, pointer *n* and *m* address registers
   - The columns of the dictionary are the address, the pointer and the symbol. At start it has $|A| + 1$ rows, the content of the i-th row is $(i, 0, a_{i-1})$. The 0th row is (0, 0, nil).
   - At start n = 0, m = $|A| + 1$, i.e. the address register points to the dictionary's end.
2. Read the next pointer *n*, exit if no such symbol exists
3. Identify the root symbol (a_x) by a backward search through the pointers stored in the dictionary registries.
4. At the address *m* the value of the pointer is set to *n*, leaving the symbol column empty for now.
5. At the m - 1-th position the symbol is set to a_x. This step is skipped when the first pointer is received.

[10] Imre Madách's The Tragedy of Man (Az ember tragédiája)

6. Decode the symbols sequence starting at the address pointed by *n*.
7. m = m + 1
8. Return to step 2.

Example 2.4

LZ code for a source of 3 symbols

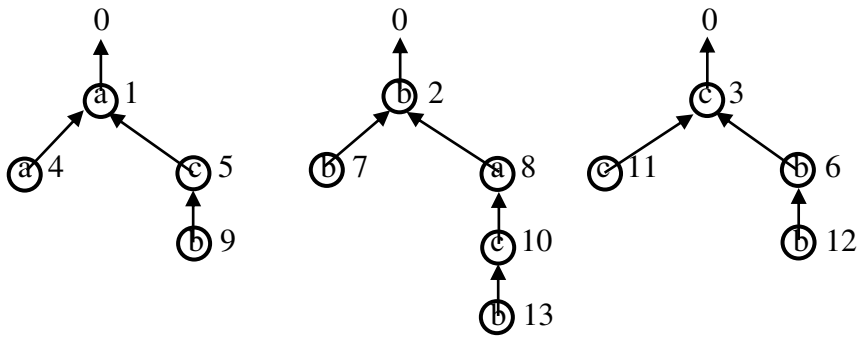
*Let the source be A = {a, b, c} and encode "*a a c b b a c b a c c b b a c b*" sequence.*

The code word sequence sent is 1 1 3 2 2 5 8 3 6 10, n = 2 (the underlined code words are encoding multiple symbols).

The dictionary built when encoding:

Address (m)	*pointer (n)*	*symbol*
0	0	nil
1	0	a
2	0	b
3	0	c
4	1	a
5	1	c
6	3	b
7	2	b
8	2	a
9	5	b
10	8	c
11	3	c
12	6	b
13	10	b, *and n = 2*

The graphical representation of the dictionary's content (the "forest"):



Practical considerations

If addresses are stored on b bits (length of register m is b) then the maximum length of the dictionary is 2^b. So a "too small" b might cause problems during encoding, yet using a big one results in unnecessary high usage of sending/storing capacity, as the message sent/stored is made of the addresses in the dictionary.

2.5. Arithmetic Coding

This coding method is similar to Huffman coding as it requires a priori knowledge of the source distribution, and is similar to LZ coding because it assigns a single code word (a real number) to a sequence of source symbols. Its practical applications include image and video compression.

The basic idea behind the algorithm is that parts of the [0, 1) interval are assigned to the source symbols, based on their probabilities. For example, if A = {a, b, c} and P(A) = {0.4, 0.4, 0.2}, then we get 3 intervals: [0, 0.4), [0.4, 0.8) and [0.8, 1). The "first" symbol (e.g. b) is matched to the proper subinterval (in case of b it is [0.4, 0.8)). Then this subinterval serves as the basis interval and it is divided into more subintervals in the same manner as in the previous step (so we get [0.4, 0.56), [0.56, 0.72) and [0.72, 0.8)). From these three, the second symbol (e.g. c) is matched to its interval ([0.72, 0.8)), and so on. As the number of processed symbols increases, the size of the actual subinterval decreases. Finally, if no symbol is left to be encoded, an arbitrary real number is chosen from the last interval to represent the whole string which is being encoded.

Example 2.5
Let A = {a, b, c} = {0.5, 0.25, 0.25}
Then the subinterval for the symbol sequence "b a a" *is* [0.5, 0.5625), and any number from this interval could be selected.

When selecting a final number from the subintervals, it is advisable to choose one that can be transmitted via binary channels or storage devices at a low cost. So we should choose the "shortest" (least number of digits) binary fraction. From the example above, for [0.72, 0.8), it would be 0.75, which can be transmitted as "0.11" (or by saving the starting "0", just as "11").

Example 2.6
A = {b, a} = {1/3, 2/3}. *Identify the interval boundaries for symbol sequences of length 3!*

Symbol sequence	*interval*	*chosen number*	*binary fraction's length*
bbb	[0, 0.0366)	0.03125	5
bba	[0.0366, 0.11)	0.06125	4
bab	[0.11, 0.183)	0.125	3
baa	[0.183, 0.33)	0.25	2
abb	[0.33, 0.4033)	0.375	3
aba	[0.4033, 0.55)	0.5	1
aab	[0.55, 0.7)	0.625	3
aaa	[0.7, 1)	0.75	2

The example tries to demonstrate that the more probable a symbol sequence is, the wider the assigned interval and shorter the generated code word is.

Naturally, the decoding procedure is then started by identifying the symbol encoded first. It can be noticed that the effect of encoding a symbol is adding a number to the interval bounds, which might cause a change in more than one digits of the chosen binary fraction. This way arithmetic coding distributes the code bits between the symbols, so it is unaffected by the quantizing effect Huffman coding has.

Practical considerations
When decoding, a problem might arise: if symbol sequences of a given length are used every time in encoding, then what guarantees that the resulting variable-length code is decodable? Or if we use code words of the same length instead (i.e. we stop decoding, when a specified binary fraction is reached) then how does the decoder know when to stop? The solution is applying a

special STOP symbol: it has to be treated in the same manner as other elements of the source alphabet, i.e. a probability has to be assigned to it.

Another problem could be caused by the finite numerical precision (arithmetic overflow when calculating interval bounds), that is, we cannot encode a symbol sequence of truly arbitrary length into one code word. The "infinite-precision arithmetic" can be simulated by computer science techniques. Suppose that somehow we know that the first few digits of the interval bounds will not change. For example, if we are between 0.2 and 0.4 – so both are smaller than 0.5 – then the first digit is surely 0. So we can already send this fixed digit, and can continue the calculation with the rest.

2.6. Evaluating Source Coding Methods

The main purpose of the three methods mentioned before was to "**compress**" the sources without any loss, i.e. identifying such binary code words which have their average code length as close to the theoretical minimum (source entropy) as possible. In an optimal case, when this minimum is achieved, every symbol of the compressed message contains 1 bit information on *average*.

The oldest and simplest method is **Huffman coding**. Its advantage is that it can be implemented with simple and fast hardware (code words have to be read from a table) and that by every source extension we get closer to achieving optimal code. A drawback is that for small alphabets or those which have "extreme" source distributions, the generated codes might have low efficiency. It is because after constructing the dictionary, *every time* a given symbol appears, the same code word made of the same *integer* number of binary symbols is sent to the channel (quantizing effect). So the efficiency is not improved as the length of the message is increased. Yet, if the source is memoryless, by source extension the code can approach optimality, but only at the price of increasing the cost and operation time of the encoder used.

As an alternative to Huffman coding, in case of "too small" alphabets, there is **arithmetic coding**, which also uses a priori knowledge of source distribution. Its advantage is that by dividing code bits the quantizing effect is eliminated and it generates an asymptotically optimal code. But a trade-off is its relatively higher complexity.

Finally, we have **LZ-coding** with its completely different approach that needs no knowledge of the source distribution. However, in order to achieve high efficiency, a symbol sequence of adequate length is required.

It is important to note that *there is no* lossless compression algorithm that could *always* encode a binary source's symbol sequence of arbitrary length N by using fewer symbols than N (not even by 1 less, i.e. with at most N-1 binary symbols). This can be easily understood with the help of the pigeonhole principle (see Appendix A2). The statements on code efficiency are only meant in ***general***, as an *average* evaluation.

Example 2.7
In example 2.3 using naïve coding L(A) = 2, with Huffman coding L(A) = 1.7. But if the source emits only a_2 and a_3 for a long time period, then the Huffman coding results in 3 channel uses for a symbol to transmit, while naïve needs only 2.
How is this possible? It is simple: we design codes for general *(probable) cases, not for special and rare symbol sequences.*

As already mentioned, in source coding we suppose that the channel is not affected by noise and that no information is lost during the transmission process (the code is decodable). The main application areas of **lossy** encoding are image, audio and video compression, where the special features of human perception are exploited. (These, however, are not mentioned in this syllabus.)

In case we want to take into account the channel noise, we need to move on to the field of **channel coding**.